

EXPO-2026 키오프 실습 정리

인공지능소프트웨어학과 / 20242517 / 서유민

1. 실습 개요

첨부된 「cli_github_사용법」 자료를 바탕으로, 터미널(CLI) 환경에서 AI 에이전트를 다루는 방법과 Git/GitHub을 이용해 코드를 버전 관리하고 팀과 공유하는 방법을 실습했다. 팀 저장소 **DMU-AILAB/EXPO-2026**을 로컬 PC에 clone하고, 실습용 브랜치(kickoff/20242517)를 생성한 뒤, Antigravity(Google의 에이전트 우선 IDE)를 이용해 AI 에이전트에게 자연어로 작업을 지시하고 실행 과정을 직접 확인했다. 이후 변경 사항을 commit, push하여 main 브랜치로 Pull Request를 생성하는 과정까지 Git/GitHub 기본 워크플로우 전체를 수행했다.

핵심 도구 3가지

도구	역할
Claude Code	터미널 안에서 대화하듯 코딩하는 AI 에이전트(Anthropic). 명령어 claude로 실행하는 CLI 기반 도구.
Antigravity	여러 AI 에이전트에게 작업을 맡기고 결과를 검토하는 구글의 에이전트 우선(agent-first) IDE. GUI 기반.
Git & GitHub	코드 변경 이력을 기록(Git)하고, 온라인에 저장·공유(GitHub)하는 버전 관리 도구.

한 줄 요약 — AI 도구가 코드를 만들면, Git·GitHub가 그것을 안전하게 저장·공유한다. 둘은 짝꿍이다.

2. AI 에이전트 CLI란?

CLI(Command-Line Interface)는 마우스 클릭 대신 텍스트 명령어(터미널)를 통해 컴퓨터를 조작하는 방식이다. **AI 에이전트 CLI**는 그 터미널 환경에서 사람이 자연어로 지시하면 AI가 직접 코드를 읽고·작성·수정하고, 필요한 터미널 명령어 실행까지 대신 수행해 주는 개발 도구를 뜻한다.

- "이 버그 고쳐줘"라고 쓰면 AI가 알아서 관련 코드를 찾아 수정
- 여러 파일을 한 번에 읽고 프로젝트 전체 맥락을 이해
- 테스트 실행, git commit 등 터미널 명령까지 (허락을 받으며) 대신 처리

터미널 필수 이동 명령어

명령어	설명
pwd	현재 디렉토리(내 위치) 확인
ls	현재 폴더의 파일 및 폴더 목록 보기
cd [폴더명]	지정한 폴더로 이동 (예: cd EXP0-2026)
cd ..	상위 폴더로 이동
mkdir [폴더명]	새로운 폴더 생성

3. Claude Code — 터미널에 사는 코딩 파트너

말로 지시

"테스트 추가해줘"처럼 자연어로 요청하면 코드를 직접 작성·수정한다.

프로젝트 이해

폴더 전체를 읽어 맥락을 파악하고, 관련 파일을 스스로 찾는다.

명령 실행

테스트 실행·git commit 같은 터미널 작업도 허락을 받고 대신 처리한다.

설치 및 첫 실행

```
# 1) Node.js 설치 여부 확인
node --version

# 2) Claude Code 전역 설치
npm install -g @anthropic-ai/claude-code

# 3) 설치 확인
claude --version

# 4) 작업 폴더로 이동 후 실행 (처음엔 브라우저로 로그인)
cd my-project
claude
```

로그인이 끝나면 터미널로 돌아와 대화창이 뜨고, 이후 한국어로 자유롭게 질문·지시할 수 있다. 지시할 때는 **파일명을 꼭 집어 목표와 조건을 구체적으로** 말해야 결과가 정확하다.

× 막연한 지시

"코드 좀 좋게 만들어줘" — 무엇을·어디서·어떻게인지 알 수 없어 결과가 엉뚱해질 수 있음

✓ 구체적 지시

"src/login.js의 로그인 함수에 이메일 형식 검사를 추가하고, 실패 시 에러 메시지를 보여줘"

자주 쓰는 명령어

명령어	설명
/help	사용 가능한 명령 목록 보기
/init	프로젝트 안내 파일 자동 생성
/clear	대화 맥락 초기화 (새 작업 시작)
/undo	방금 한 변경 되돌리기
/exit, Esc	Claude Code 종료 / 작업 중단

4. Antigravity — 에이전트에게 맡기는 IDE

구글이 만든 AI IDE(코드 편집 프로그램)로, VS Code를 바탕으로 만들어져 화면이 익숙하다. Claude Code와의 차이점은 "에이전트 우선(agent-first)" 방식이라는 점이다 — 사람이 직접 타이핑하기보다, 여러 AI 에이전트에게 작업을 맡기고 그 결과를 검토하는 방식으로 동작한다.

Editor View

직접 코드를 보고 고치는, VS Code와 유사한 익숙한 화면

Manager View

여러 에이전트를 동시에 지켜보는 "관제탑" 화면

Artifacts

계획·변경 내역을 문서로 남겨 검토하기 쉽게 해주는 기능

에이전트가 일하는 순서

- 목표 입력** — "할 일 목록 앱 만들어줘"처럼 원하는 목표를 적는다
- 계획(Plan) 검토** — 에이전트가 작성한 계획 문서를 확인 후 승인한다
- 실행 & 검증** — 코드를 작성하고 스스로 테스트하며 결과물을 남긴다
- 피드백** — 문서에 댓글 달듯 의견을 주면 반영한다

설치 시에는 입문자에게는 **Agent-Assisted** 모드(중요한 변경은 직접 승인)가 권장된다. 프리뷰 단계라 사용량 제한이 있을 수 있으므로, 중요한 작업은 결과를 꼭 직접 확인해야 한다.

5. Git과 GitHub의 차이

구분	설명	비유
Git	내 컴퓨터(로컬 PC)에서 코드의 변경 이력을 기록하는 버전 관리 프로그램	게임의 세이프 포인트
GitHub	Git으로 관리한 코드를 온라인에 올려 백업·공유하고 협업하는 웹 플랫폼	코드 전용 클라우드 드라이브

Git 기본 워크플로우 4단계

```
[작업 폴더 (Working Directory)] | ▼ git add . [스테이징 영역 (Staging Area)] | ▼ git commit -m "메시지" [로컬 저장소 (Local Repo)] | ▼ git push origin [브랜치명] [원격 저장소 (GitHub)]
```

반대 방향 — `git clone`은 GitHub 코드를 내 PC로 통째로 복사, `git pull`은 바뀐 부분만 받아온다.

실전 필수 Git 명령어

명령어	설명
<code>git clone <url></code>	GitHub 저장소를 내 PC로 복사
<code>git checkout -b [브랜치명]</code>	신규 브랜치 생성 및 이동
<code>git status</code>	지금 뭐가 바뀌었는지 확인 (헷갈리면 항상 이것부터)
<code>git add .</code>	바뀐 파일을 커밋 대상(스테이징)에 담기
<code>git commit -m "메시지"</code>	변경 내용을 하나의 기록으로 저장
<code>git push origin [브랜치명]</code>	기록을 GitHub 원격 저장소에 올리기
<code>git pull</code>	원격 저장소의 바뀐 부분만 받아오기

6. 실전 수행 명령어

본 실습에서 실제로 EXPO-2026 저장소에 대해 순서대로 수행한 명령어이다.

```
# 1. 원격 저장소 복사해오기
git clone https://github.com/DMU-AILAB/EXPO-2026.git
cd EXPO-2026

# 2. 신규 브랜치 생성 및 이동
git checkout -b kickoff/20242517

# 3. 상태 확인
git status

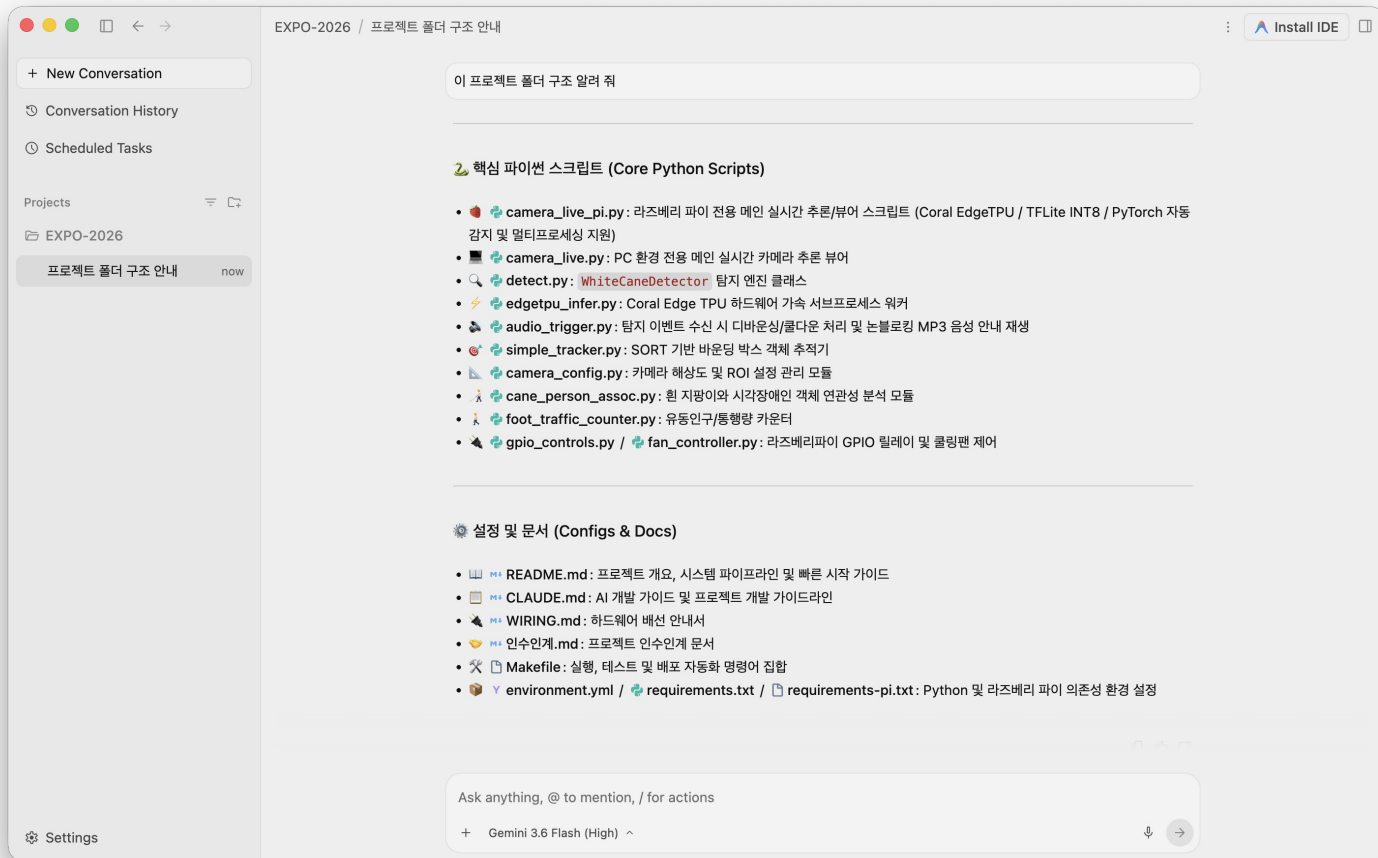
# 4. 변경사항 저장 후보에 올리기
git add docs/20242517_서유민.pdf

# 5. 변경사항 기록하기
git commit -m "docs: EXPO 킥오프 실습 요약 PDF 업로드"

# 6. 원격 저장소로 올려서 PR 준비하기
git push origin kickoff/20242517
```

7. Antigravity로 진행한 실습 캡처

Antigravity IDE에서 EXPO-2026 프로젝트를 열고, AI 에이전트에게 프로젝트 폴더 구조 설명을 요청한 화면이다. 에이전트가 `camera_live_pi.py`, `detect.py` 등 핵심 스크립트와 `README.md`, `requirements.txt` 등 설정/문서 파일의 역할을 정리해 응답했다.



Antigravity — EXPO-2026 프로젝트 폴더 구조 안내 (AI 에이전트 응답 화면)

8. 실전 워크플로우 요약 (CLI + GitHub)

1. 코드 받기 — GitHub에서 `git clone`
2. AI로 작업 — Claude Code / Antigravity로 기능 추가·수정
3. 직접 확인 — 결과가 맞는지 실행·검토 (AI 결과는 항상 검증)
4. 기록 — `git add` → `git commit`
5. 올리기 — `git push` → GitHub에서 Pull Request 생성

자주 하는 실수와 팁

commit 없이 push 시도

`add` → `commit`을 먼저 해야 올릴 게 생긴다.

엉뚱한 폴더에서 명령 실행

항상 `pwd`로 내 위치부터 확인한다.

AI 결과는 꼭 검토

AI도 틀릴 수 있다. 실행해 보고 이해한 뒤 커밋한다.

자주, 작게 커밋

한 번에 몰아서 하지 말고, 의미 단위로 자주 저장하면 되돌리기 쉽다.

9. Pull Request 생성

push가 완료된 후 GitHub 저장소 페이지에서 **base: main** ← **compare: kickoff/20242517**로 Pull Request를 생성하여 팀 리뷰를 요청했다.